Garmin Connect Activity Import Integration

Context

WearWise already has an "Import from Apple Health" feature that fetches recent workouts and prefills the activity logging form. The user wants the same capability for Garmin Connect, so users with Garmin devices can import their activities too.

Key difference from Apple Health: Garmin requires OAuth web-based authentication (not a system permission dialog) and REST API calls (no native SDK). Users simply sign in with their personal Garmin Connect account — standard OAuth flow, low friction.

Developer registration prerequisite: You (the app developer) must register once at the Garmin Developer Portal to get API credentials (consumer key/secret). These get embedded in the app. Credentials are placeholders in code until registration is approved (~2 business days).

Important Garmin API compliance notes:

- Must register at Garmin Developer Portal (approval ~2 business days, legal entity required)
- License is limited, non-exclusive, non-transferable, revocable
- Data usage restricted to formatting and displaying in the app
- Garmin may review/approve the app before publication
- Rate limits: evaluation tier allows 100 calls/min, 200 calls/user/day
- **OAuth variant**: Garmin's API may use OAuth 1.0a (not 2.0+PKCE) — will implement OAuth 1.0a as that's what the Garmin Connect Developer Program actually uses, with clear abstraction so it can be adjusted after registration

Changes

1. New: `Services/GarminConnectService.swift`

Mirrors `HealthKitService.swift` structure:

- **`GarminWorkout` struct** — same interface as `HealthWorkout`: `id`, `activityType` (String), `startDate`, `endDate`, `duration`, computed `durationMinutes`, `matchedActivityName`, `displayName`
- **`GarminConfig` struct** — placeholder `consumerKey`, `consumerSecret`, callback URL scheme (`wearwise`), API endpoints
- **`@Observable class GarminConnectService`** with:
  - Same `AuthStatus` enum (`.notDetermined`, `.authorized`, `.denied`)
  - Same observable properties: `workouts`, `isLoading`, `errorMessage`, `authorizationStatus`
  - OAuth flow via `ASWebAuthenticationSession` — opens Garmin login in web sheet, handles callback
  - Keychain token storage (Security framework: `SecItemAdd` / `SecItemCopyMatching` / `SecItemDelete`) with `kSecAttrAccessibleWhenUnlockedThisDeviceOnly`
  - `requestAndFetch()` — authenticate if needed, then fetch activities
  - `fetchRecentActivities(limit:)` — GET to Garmin activities API, parse JSON with Codable
  - Token refresh on 401 responses
  - `disconnect()` — clear tokens, reset state
  - Static mapping methods matching HealthKitService pattern:
    - `activityName(for:)` — maps Garmin types ("running", "hiking", "cycling", etc.) → app ActivityType names
    - `displayName(for:)` — human-readable names

    - `icon(for:)` — SF Symbols
- Wrapped in `#if os(iOS)`

2. New: `Views/iOS/GarminWorkoutPickerView.swift`

Mirrors `HealthWorkoutPickerView.swift` structure:

- Takes `@Bindable var service: GarminConnectService` + `onSelect: (GarminWorkout) -> Void`
- Same state handling: loading, error, notDetermined (shows "Sign in to Garmin Connect" button), denied, empty, success
- Workout list grouped by date (same `groupedByDate` pattern)
- `GarminWorkoutRow` — same layout as `WorkoutRow` but with Garmin-blue accent (`Color(red: 0.0, green: 0.63, blue: 0.89)`) instead of pink
- Nav title: "Garmin Connect Activities"
- `#if os(iOS)` wrapped

3. Modified: `Views/Experience/LogExperienceView.swift`

In `ActivityStepView` :

- Add `@State private var showingGarminImport = false` and `@State private var garminService = GarminConnectService()` (inside `#if os(iOS)`)
- Add "Import from Garmin Connect" button (with `figure.run.circle` icon, Garmin-blue tint) in the same Section as the Apple Health button
- Add `.sheet(isPresented: $showingGarminImport)` presenting `GarminWorkoutPickerView`
- Add `applyGarminWorkout(_:)` — identical logic to `applyWorkout(_:)` since `GarminWorkout` exposes the same interface

4. Modified: `Info.plist`

Add `CFBundleURLTypes` with scheme `wearwise` for OAuth callback redirect (`wearwise://garmin-callback`).

5. Modified: `Views/Settings/SettingsContent.swift`

Add "Connected Accounts" section showing Garmin connection status with disconnect button — since unlike Apple Health (system-managed), Garmin auth is app-managed and users need a way to sign out.

Files to modify/create

File	Action
`WearWise/WearWise/Services/GarminConnectService.swift`	Create
`WearWise/WearWise/Views/iOS/GarminWorkoutPickerView.swift`	Create
`WearWise/WearWise/Views/Experience/LogExperienceView.swift`	Modify (ActivityStepView)
`WearWise/WearWise/Info.plist`	Modify (add URL scheme)
`WearWise/WearWise/Views/Settings/SettingsContent.swift`	Modify (add connected accounts)

Key reference files

- `Services/HealthKitService.swift` — primary pattern to follow
- `Views/iOS/HealthWorkoutPickerView.swift` — picker UI pattern
- `Views/Experience/LogExperienceView.swift:255-331` — integration point (ActivityStepView)

- `Models/ActivityType.swift` — activity name matching

Verification

1. Build the project — all new code is `#if os(iOS)`, macOS should be unaffected
2. Run on iOS simulator — "Import from Garmin Connect" button should appear alongside Apple Health
3. Tapping the button should show the GarminWorkoutPickerView with "Sign in to Garmin Connect" state
4. OAuth flow will only work with real Garmin developer credentials (placeholder until registration approved)
5. Verify the settings disconnect flow works